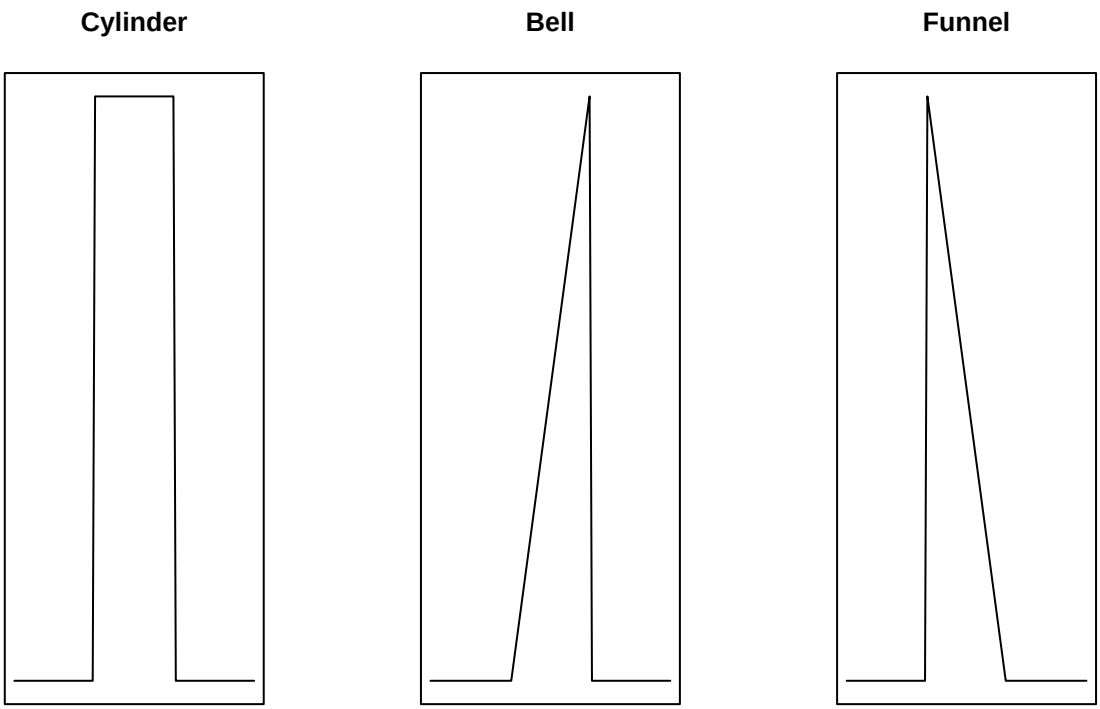
Clustering time series databases

Usue Mori

25 de febrero de 2026

In this tutorial we will cluster time series databases using different techniques. As an example, we will use the CBF (Cylinder-Bell-Funel) database from the UEA & UCR Time Series Classification Repository (`https://www.timeseriesclassification.com`). This database contains series from 3 classes, each of which is represented by one of the shapes shown in the following figure.



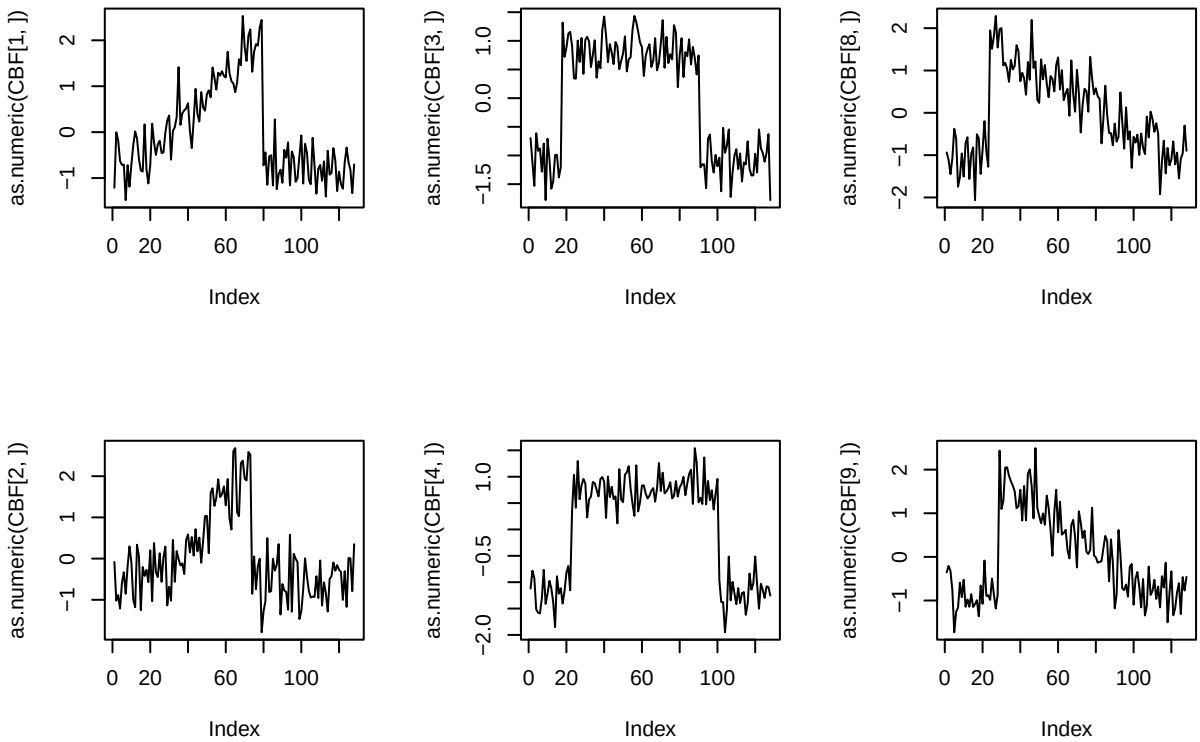
The dataset is saved in a `data.frame`, where each row corresponds to an instance (a time series, in this case). Additionally, since this dataset is initially designed for supervised classificatio, the class of each time series is also available, in the last column of the `data.frame`.

Let's load the data and separate the class variable, we will not use it for clustering:

```
CBF <- read.table("./Data/CBF.txt", dec=".", sep=";")
CBFtipo <- CBF[, ncol(CBF)]
CBF[, ncol(CBF)] <- NULL
```

As stated, each time series has one of the three shapes shown in the previous figure. However, for each time series the shape is situated in a different location (shifted), it can be streched or compressed (warped) and it contains noise. Let's see some examples:

```
par(mfrow=c(2,3))
plot(as.numeric(CBF[1,]), type="l")
plot(as.numeric(CBF[3,]), type="l")
plot(as.numeric(CBF[8,]), type="l")
plot(as.numeric(CBF[2,]), type="l")
plot(as.numeric(CBF[4,]), type="l")
plot(as.numeric(CBF[9,]), type="l")
```



1. Distance based clustering

In this section, we will first learn about distance computation in R. Then, we will use these distance to cluster a time series database.

1.1. Distance computation using *proxy* and *dtw*

Time series distances are implemented in different packages in R. In this lab, we will use packages *proxy* and *dtw*. First, we will focus on calculating the distance between two time series (the first two of the dataset, for example):

```
library(proxy)
library(dtw)
```

We will first calculate the Euclidean distnce:

```
#Save the first two series of the database in a data frame:
data <- CBF[c(1,2),]
```

```
#Compute Euclidean distance
dist(data, method="euclidean")
```

```
         367
323 10.15657
```

Similarly we can also compute the DTW distance. Note that, when using function `dtw` we need to choose option `step.pattern = symmetric1` so that the procedure is the one explained in class:

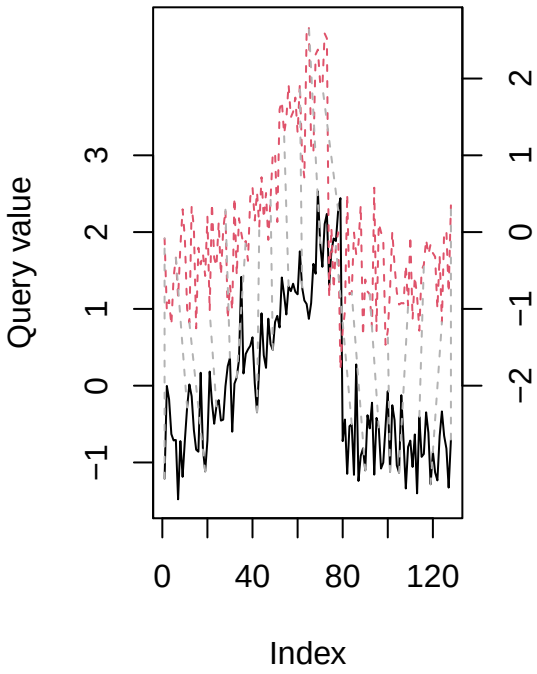
```
# Compute DTW distance
dist(data, method="dtw", step.pattern = symmetric1)
```

```
        367
323 41.4061
```

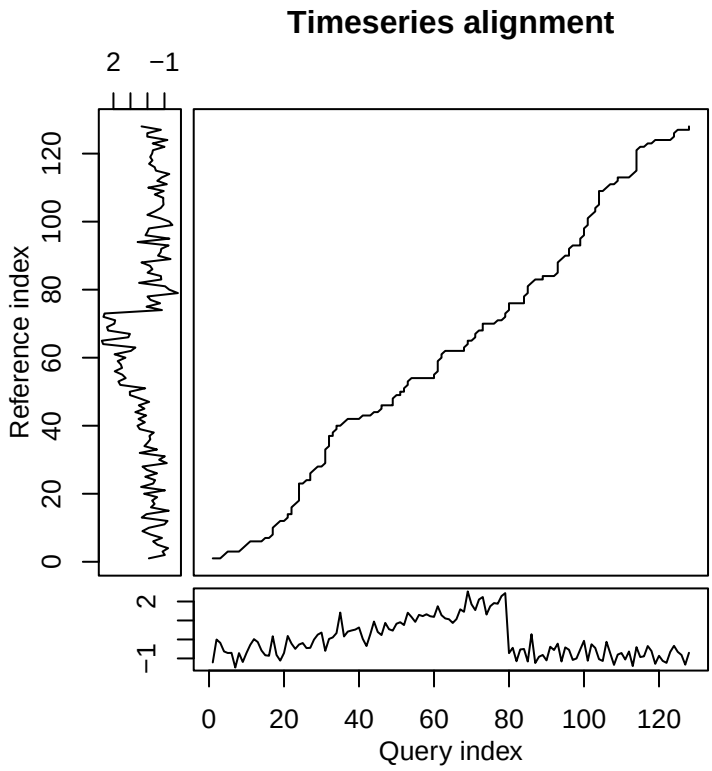
In the case of the DTW distance, we can also get additional information about the aligment between the two time series:

```
# Perform alignment
alignment <- dtw(x=as.numeric(CBF[1,]), y=as.numeric(CBF[2,]), keep.internals=TRUE)

# Display the optimal path
par(mfrow=c(1,2))
plot(alignment, type = "twoway", off=2, match.lty=2, match.indices=20)
```



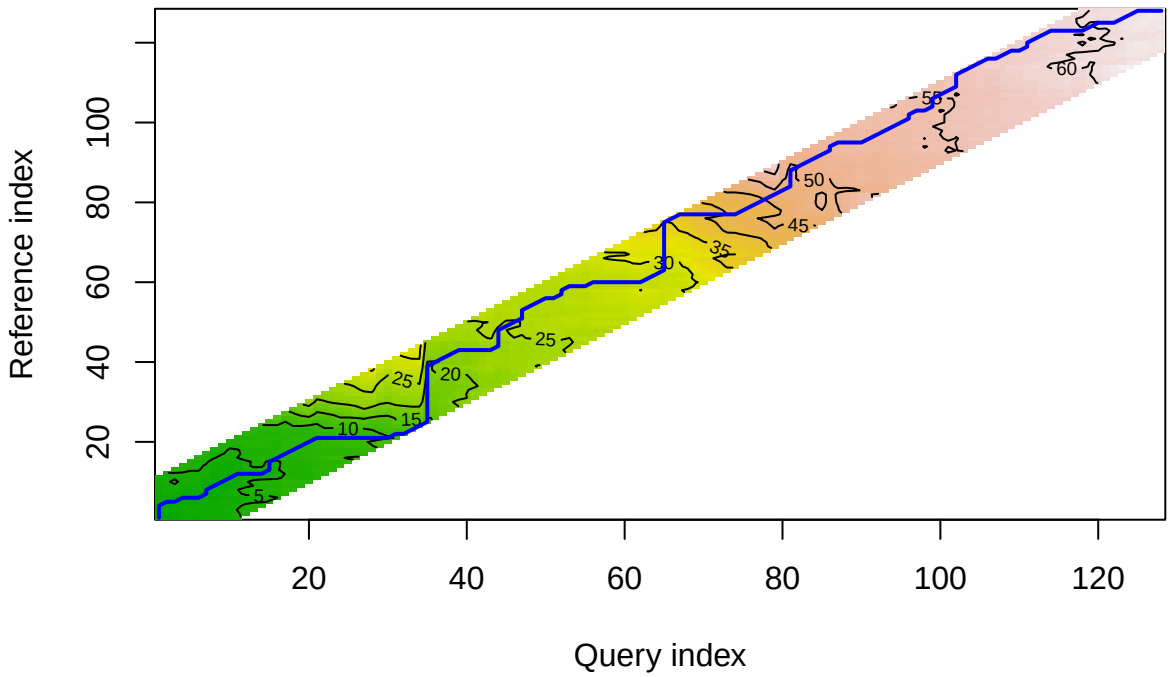

```
plot(alignment, type = "threeway")
```



We can also calculate variants such as the Sakoe-Chiba banded distance:

```
# Compute DTW distance with a Sakoe-Chiba band of size 10
alignment_windowed <- dtw(x=as.numeric(CBF[1,]), y=as.numeric(CBF[8,]), step.pattern = symmetric1,
     window.type="sakoechiba", window.size=10, keep.internals=TRUE)

plot(alignment_windowed, type="density")
```



1.2. Clustering using distances

Now that we know how to calculate distances, to cluster a dataset using a specific time series distance, first, we need to calculate all the pairwise distances between all the time series in the dataset:

```
d1 <- dist(CBF, method="euclidean")
d2 <- dist(CBF, method="dtw", step.pattern=symmetric1)
```

As we saw in class, when we use specific distance measures for time series which are not metrics, we can not use algorithms such as k-means. In these cases it is more suitable to use k-medoids or hierarchical clustering. Let's start with k-medoids:

```
library(cluster)
#Euclidean distance with k=3
clus1.euc <- pam(d1,  k=3, diss=TRUE)
##DTW with k=3
clus1.dtw <- pam(d2,  k=3, diss=TRUE)
```

An object of type *pam* saves diverse information about the clustering result (medoids, clustering, etc.). In order to access the cluster assigments for each instance we do the following:

```
#Euclidean distance with k=3
clus1.euc.groups <- pam(d1,  k=3, diss=TRUE)$clustering
##DTW with k=3
clus1.dtw.groups <- pam(d2,  k=3, diss=TRUE)$clustering
```

Similarly we can also apply hierarchical clustering, implemented in the `hclust` package. In this case, we must provide a distance matrix as input:

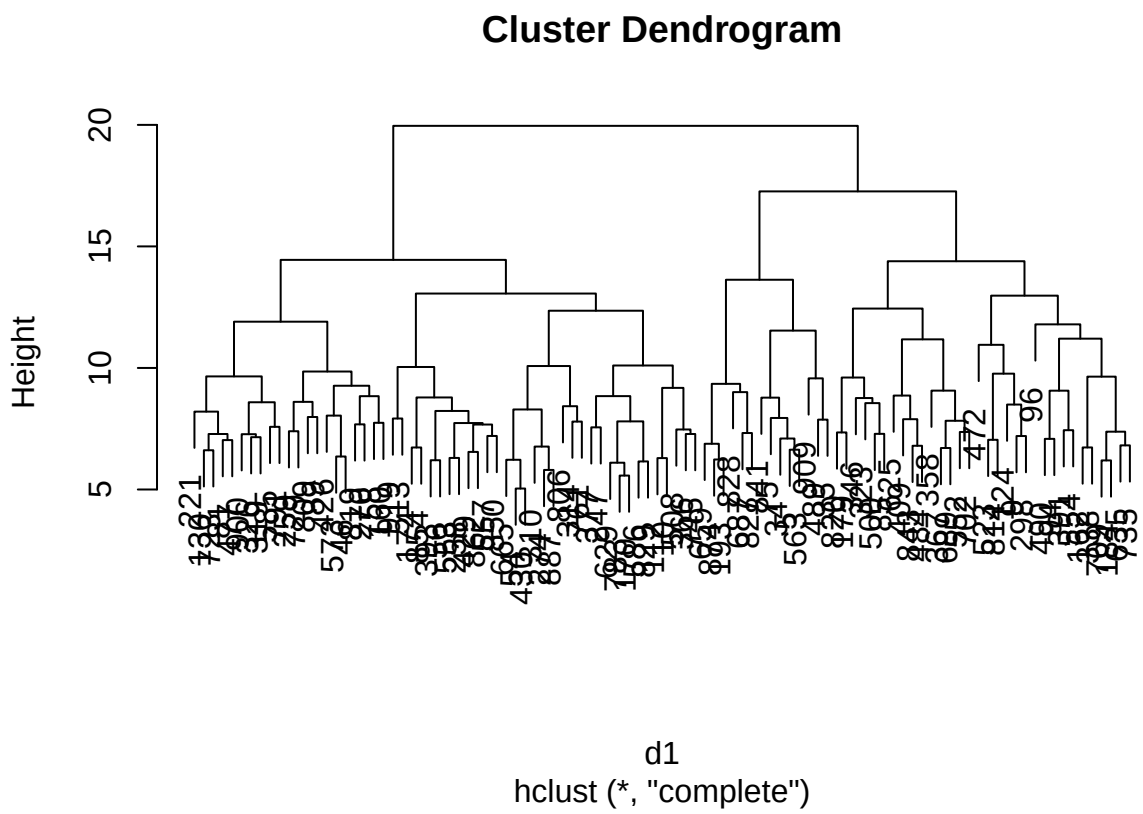
```
#Euclidean distance
clus2.euc <- hclust(d1,  method="complete")
#DTW
clus2.dtw <- hclust(d2,  method="complete")
```

In this case, since we know that the number of natural groups in the data is 3, we will cut the dendogram at this level as follows to obtain the cluster asignations of the time series in the database:

```
clus2.euc.groups<-cutree(clus2.euc, k=3)
clus2.dtw.groups<-cutree(clus2.dtw, k=3)
```

However, if we do not know the value of k, we can represent the dendogram and select the most suitable k by analyzing the graphic:

```
#Euclidean distance
plot(clus2.euc)
```



2. Feature based clustering

Package `tsfeatures` from R, provides the implementation of a large variety of features to describe time series. We will use this package to obtain a feature based representation of our database:

```
library(tsfeatures)
```

```
FALSE Registered S3 method overwritten by 'quantmod':
FALSE   method            from
FALSE   as.zoo.data.frame zoo
```

```
CBF <- lapply(seq_len(nrow(CBF)), function(i) as.numeric(CBF[i,]))
CBFfeatures <- tsfeatures(CBF)
head(CBFfeatures)
```

```
# A tibble: 6 x 16
  frequency nperiods seasonal_period trend     spike linearity curvature  e_acf1
      <dbl>    <dbl>           <dbl> <dbl>     <dbl>     <dbl>     <dbl>   <dbl>
1         1        0               1 0.811   7.15e-6   -2.12       -7.37  0.0591
2         1        0               1 0.711   9.43e-6   -1.16       -5.48 -0.108
3         1        0               1 0.870   2.64e-6   -3.58       -8.10  0.0226
4         1        0               1 0.891   2.13e-6   -0.888      -9.31  0.0530
5         1        0               1 0.755   6.96e-6   -2.25       -7.16 -0.0621
6         1        0               1 0.862   2.48e-6   -0.0489     -9.07  0.171
# i 8 more variables: e_acf10 <dbl>, entropy <dbl>, x_acf1 <dbl>,
#   x_acf10 <dbl>, diff1_acf1 <dbl>, diff1_acf10 <dbl>, diff2_acf1 <dbl>,
```

```
#   diff2_acf10 <dbl>
```

As can be seen, each row of matrix `CBFfeatures` refers to one time series from our database and each column represents a feature. To see the entire list and explanation of the features included in `tsfeatures` you can go to the following vignette:

`https://cran.r-project.org/web/packages/tsfeatures/vignettes/tsfeatures.html`

Once we have a feature based representation of our time series, we have eliminates the temporal components and can apply any common clustering algorithm such as k-means with standard distance measures such as the Euclidean distance. We can do this as follows:

```r
clusfeatures <- kmeans(CBFfeatures, centers=3)$cluster
```

3. Evaluation

In most non-supervised classification problems, we do not have the labels of the instances, so, the most common method to evaluate the results is by visual inspection. However, when we do have information about the ground truth, we can use measures such as the F-measure implemented in the following function:

```r
# F measure to evaluate clustering results
F <- function(clus, true) {
  tab1 <- 2 * table(clus, true)
  tab2 <- outer(rowSums(tab1), colSums(tab1), `+`)
  result <- 1 / length(true) * sum(colSums(tab1) * apply((tab1 / tab2), 2, max))
  return(result)
}
```

In this case,

```r
F(clus1.euc.groups, CBFtipo) # K-medoids with Euclidean distance
```

```
[1] 0.5726553
```

```r
F(clus1.dtw.groups, CBFtipo) # K-medoids with DTW
```

```
[1] 0.9694604
```

```r
F(clus2.euc.groups, CBFtipo) # Hierarchical clustering with Euclidean distance
```

```
[1] 0.6099476
```

```r
F(clus2.dtw.groups, CBFtipo) # Hierarchical clustering with DTW
```

```
[1] 0.579534
```

```r
F(clusfeatures, CBFtipo) # K-means with features
```

```
[1] 0.5480833
```

As you can see, in this case, the best results (based on the ground truth) are obtained with the K-medoids algorithms and the DTW distance.

NOTE: You can find more evaluation metrics for clustering in the following reference:

S. Wagner and D. Wagner. (2007). Comparing Clusterings - Anoverview. Universitat Karlsruhe (TH), Karlsruhe, Germany, Tech.Rep. 2006-04 [Online]. Available: `http://digbib.ubka.uni-karlsruhe.de/volltexte/1000011477/bigskip`

4. Exercises

- Given these two time series: $X_t = (1, 0, 2, 3, 1, 2)$, $Y_t = (0, 1, 2, 1, 0, 3)$.
  - Calculate the DTW distance between these two time series by hand and obtain the optimal aligment between them
  - Repeat the exercise using functions `dtw` from package `dtw` in R. Did you obtain the same solution?
  - Calculate the number of possible paths using function `countPaths`.
  - If we repeat the operation using a Sakoe-Chiba band of $r = 1$, do the results change?
  - Calculate the obtained distances with the Euclidean and Manhattan distance (calculate them by hand and using R).
- The database ECG200 (saved in file ECG200.txt), contains a set of time series that measure the electrical activity that happens during a heartbeat. The class variable (represented in the header by `V1`) takes two possible values: -1 if the heartbeat is normal y 1 if there is Myocardial Infarction.
  - Randomly choose two time series and calculate the DTW, Euclidean and Manhattan distances between them.
  - Randomly choose a 20 % of the time series (maintaining the proportion of the classes) and save them in a matrix (A). Choose another 10 % (also maintaining the class proportions) and save them in matrix B.
  - Calculate all the pairwise distances between the series in matrix A using different distance measures.
  - Calculate all the pairwise distances between series in matrices A and B (one series from each matrix) using different distance measures.
- The `Trace` database from the UEA & UCR archive is a synthetic dataset designed to simulate instrumentation failures in a nuclear power plant. Each class is dedicated to a type of failure. The structure of the database is identical to the one used in the examples above: it is a `data.frame`, each row represents a time series, and the last column is dedicated to the class variable.
  - Obtain basic information about the dataset, such as the number of classes, the length of the time series, etc. Graphically represent some of the time series. Can we discriminate between the different classes at simple sight? What type of features/distances do you think will be suitable to cluster this dataset?
  - Remove the class variable from the dataset and perform different clusterings using distance measures and features. Evaluate the results by visual inspection and by using the ground truth (the class variable) and choose the best solution, in your opinion.

5. Extra work

- Additional distances for time series are implemented in other packages, and package `TSdist` makes a compilation of them. Take a look at the contents of this package and try to use other types of features to improve your results.

NOTE: This package has been archived, so in order to install it you will have to do it from source, or using package the *remotes* package.

- Additional features for time series are implemented in other packages, and package `theft` makes a compilation of them. Take a look at the contents of this package and try to use other types of features to improve your results.

6. References

- Philippe Esling and Carlos Agon. 2012. Time-series data mining. ACM Comput. Surv. 45, 1, Article 12 (December 2012), 34 pages. DOI=`http://dx.doi.org/10.1145/2379776.2379788`
- T. W. Liao, Clustering of time series data: a survey, Pattern Recognition, vol. 38, no. 11, pp. 1857–1874, Nov. 2005.
- Anthony Bagnall, Jason Lines, Aaron Bostrom, James Large and Eamonn Keogh, The Great Time Series Classification Bake Off: a Review and Experimental Evaluation of Recent Algorithmic Advances, Data Mining and Knowledge Discovery, 31(3), 2017.
- Anthony Bagnall, Jason Lines, William Vickers and Eamonn Keogh, The UEA & UCR Time Series Classification Repository, www.timeseriesclassification.com.
- Toni Giorgino (2009). Computing and Visualizing Dynamic Time Warping Alignments in R: The dtw Package. Journal of Statistical Software, 31(7), 1-24, `doi:10.18637/jss.v031.i07`.
- U Mori, A Mendiburu, JA Lozano. Distance measures for time series in R: The TSdist package. R journal, 2016.
- tsfeatures: Time Series Feature Extraction. RJ Hyndman, E Wang, Y Kang, P Talagala. Version 0.1. url: `https://github`. com/robjhyndman/tsfeatures, 2018. 6, 2018